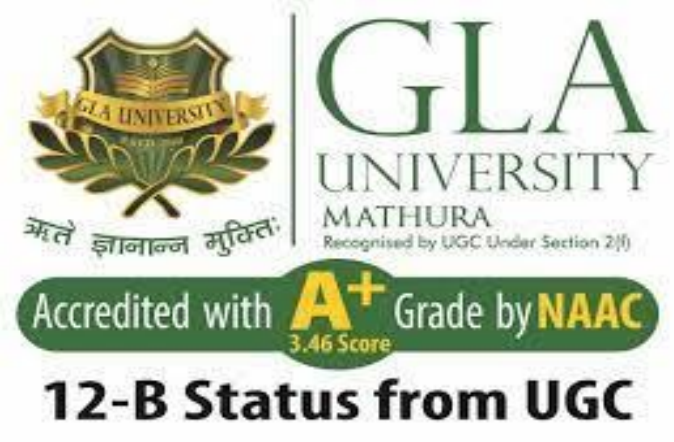




SOFTWARE PROJECT PLANNING

MS. CHESHTAA BHARDWAJ

ASSISTANT PROFESSOR, DEPT. OF CEA,

GLA UNIVERSITY, MATHURA

Module No.	Content	Teaching Hours
I	Introduction: Introduction to Software Engineering, Software characteristics, Software Crisis, Software Engineering Process. Software Development Life Cycle (SDLC) Models: Waterfall, Incremental, Iterative Enhancement, Prototype, RAD and Spiral Models. Software Requirements Engineering: Types of Requirements, Requirement Elicitation Techniques Like Interviews, FAST & QFD, Use case Approach, Requirements Analysis Using DFD, Data Dictionaries & ER Diagrams, Requirements Documentation, and SRS. Software Project Planning: Size Estimation like Lines of Code & Function Count, Cost. Estimation Models: COCOMO (Basic, Intermediate) Software Design: Cohesion & Coupling, Classification of Cohesion & Coupling, Function Oriented Design, Object Oriented Design, Structure chart. Coding: Characteristics of Coding and Coding style.	20
II	Software Metrics: Software Measurements,Token Count, Halstead Software, Measures. Software Reliability & Quality: Introduction of Mc Call's & Boehm's Quality Model, Capability Maturity Models Software Reliability Models: Basic Execution Time Model. Software Testing: Testing Fundamentals: Test Case Design, Black Box Testing Strategies, White Box Testing, Unit Testing, Integration Testing, System Testing. Introduction to Automation Testing and Testing Tools: Automated Testing Process, Framework for Automation Testing, Introduction to Automation Testing Tool. Software Maintenance: Maintenance Process Maintenance models: Belady and Lehman Model, Boehm Model Regression Testing, Software Configuration Management; Implementation, Introduction to Reengineering and Reverse Engineering. Software Risk Management: Risk Identification and Risk Analysis	20

SOFTWARE PROJECT PLANNING

- After the finalization of SRS, we would like to estimate size, cost and development time of the project.
- Also, in many cases, customer may like to know the cost and development time even prior to finalization of the SRS

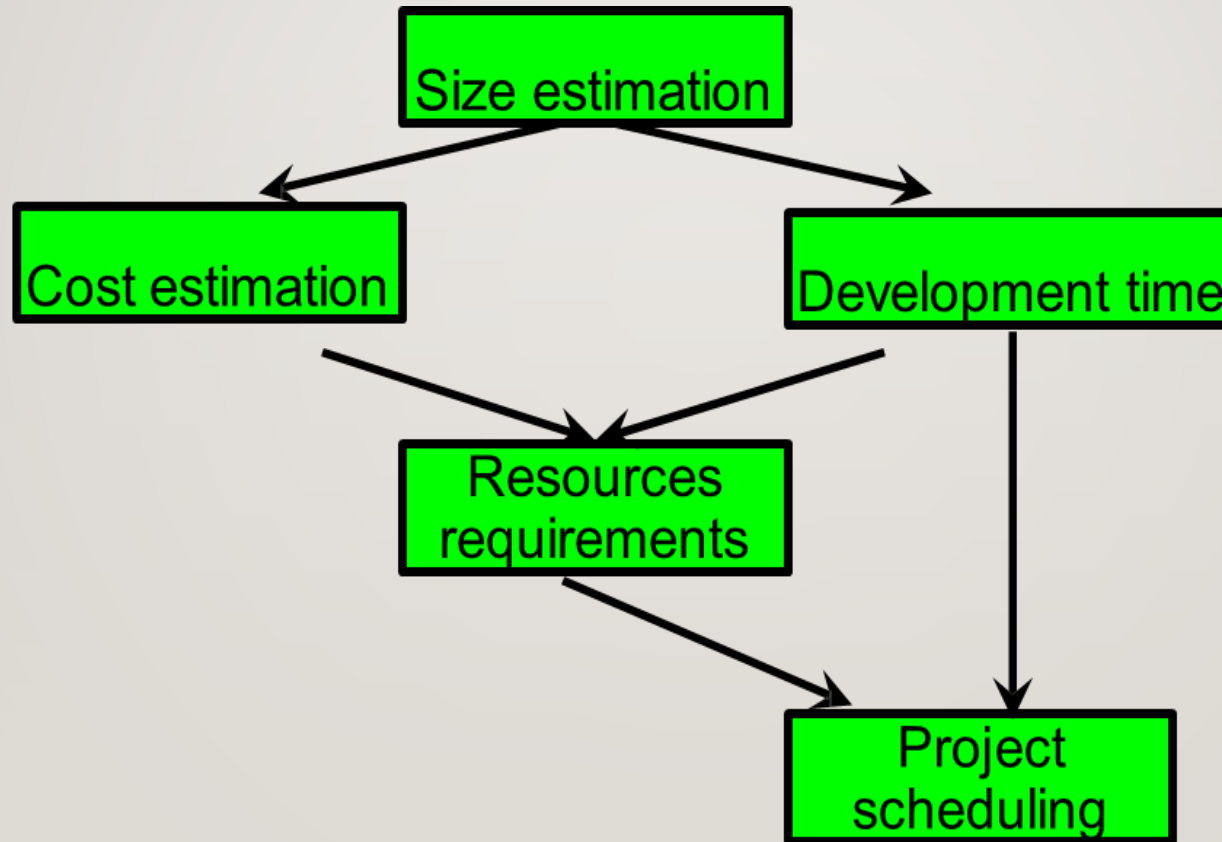
Contd.

In order to conduct a successful software project, we must understand:

- Scope of work to be done
- The risk to be incurred
- The resources required
- The task to be accomplished
- The cost to be expended
- The schedule to be followed

Activities during Software Project Planning

Software planning begins before technical work starts, continues as the software evolves from concept to reality, and culminates only when the software is retired.



SIZE ESTIMATION

- Lines of Code (LOC)
- If LOC is simply a count of the number of lines then figure shown below contains 18 LOC .
- When comments and blank lines are ignored, the program shown below contains **17 LOC**.

1.	int. sort (int x[], int n)
2.	{
3.	int i, j, save, im1;
4.	/*This function sorts array x in ascending order */
5.	If (n<2) return 1;
6.	for (i=2; i<=n; i++)
7.	{
8.	im1=i-1;
9.	for (j=1; j<=im; j++)
10.	if (x[i] < x[j])
11.	{
12.	Save = x[i];
13.	x[i] = x[j];
14.	x[j] = save;
15.	}
16.	}
17.	return 0;
18.	}

Function for sorting an array

Contd.

- Furthermore, if the main interest is the size of the program for specific functionality, it may be **reasonable to include executable statements**.
- The only executable statements in figure shown above were in lines 5-17 leading to a count of 13.
- The differences in the counts are 18 to 17 to 13.
- One can easily see the potential for major discrepancies for large programs with many comments or programs written in language that allow a large number of descriptive but non-executable statement.
- **Lines of code** as: *“A line of code is **any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line. This specifically includes all lines containing program header, declaration, and executable and non-executable statements**”.*
- **This is the predominant definition for lines of code used by researchers. By this definition, figure shown above has 17 LOC.**

FUNCTION COUNT

Alan Albrecht while working for IBM, recognized the problem in size measurement in the 1970s, and developed a technique (which he called Function Point Analysis), which appeared to be a solution to the size measurement problem.

The principle of Albrecht's function point analysis (FPA) is that a system is decomposed into functional units.

- Inputs : information entering the system
- Outputs : information leaving the system
- Enquiries : requests for instant access to information
- Internal logical files : information held within the system
- External interface files : information held by other system that is used by the system being analyzed.

SPECIAL FEATURES

- Function point approach is **independent of the language, tools, or methodologies used for implementation**; i.e. they do not take into consideration programming languages, data base management systems, processing hardware or any other data base technology.
- Function points **can be estimated from requirement specification or design specification**, thus making it possible to estimate development efforts in early phases of development.
- Function points **are directly linked to the statement of requirements**; any change of requirements can easily be followed by a re-estimate.
- Function points are **based on the system user's external view of the system, non-technical users of the software system have a better understanding of what function points are measuring**.

COUNTING FUNCTION POINTS

Functional Units	Weighting factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiries (EQ)	3	4	6
External logical files (ILF)	7	10	15
External Interface files (EIF)	5	7	10

Table 1:Functional units with weighting factors

UNADJUSTED FUNCTION POINT(UFP)

The weighting factors are identified for all functional units and multiplied with the functional units accordingly. The procedure for the calculation of Unadjusted Function Point (UFP) is given in table shown above.

Table 2: UFP calculation table

Functional Units	Count Complexity			Complexity Totals	Functional Unit Totals
External Inputs (EIs)		Low x 3	=		
		Average x 4	=		
		High x 6	=		
External Outputs (EOs)		Low x 4	=		
		Average x 5	=		
		High x 7	=		
External Inquiries (EQs)		Low x 3	=		
		Average x 4	=		
		High x 6	=		
External logical Files (ILFs)		Low x 7	=		
		Average x 10	=		
		High x 15	=		
External Interface Files (EIFs)		Low x 5	=		
		Average x 7	=		
		High x 10	=		
Total Unadjusted Function Point Count					

The procedure for the calculation of UFP in mathematical form is given below:

$$UFP = \sum_{i=1}^5 \sum_{j=1}^3 Z_{ij} w_{ij}$$

Where i indicate the row and j indicates the column of Table I

w_{ij} : It is the entry of the i^{th} row and j^{th} column of the table I

Z_{ij} : It is the count of the number of functional units of Type i that have been classified as having the complexity corresponding to column j .

Organizations that use function point methods develop a criterion for determining whether a particular entry is Low, Average or High.

Nonetheless, the determination of complexity is somewhat subjective.

$$\text{FP} = \text{UFP} * \text{CAF}$$

Where **CAF** is **complexity adjustment factor** and is equal to $[0.65 + 0.01 \times \sum F_i]$.

The F_i ($i=1$ to 14) are the **degree of influence** and are based on responses to questions noted shown in table 3 below.

Table 3 : Computing function points.

Rate each factor on a scale of 0 to 5.



Number of factors considered (F_i)

1. Does the system require reliable backup and recovery ?
2. Is data communication required ?
3. Are there distributed processing functions ?
4. Is performance critical ?
5. Will the system run in an existing heavily utilized operational environment ?
6. Does the system require on line data entry ?
7. Does the on line data entry require the input transaction to be built over multiple screens or operations ?
8. Are the master files updated on line ?
9. Is the inputs, outputs, files, or inquiries complex ?
10. Is the internal processing complex ?
11. Is the code designed to be reusable ?
12. Are conversion and installation included in the design ?
13. Is the system designed for multiple installations in different organizations ?
14. Is the application designed to facilitate change and ease of use by the user ?

FUNCTIONS POINTS MAY COMPUTE THE FOLLOWING IMPORTANT METRICS:

Productivity = FP / persons-months

Quality = Defects / FP

Cost = Rupees / FP

Documentation = Pages of documentation per FP

- These metrics are controversial and are not universally acceptable.
- There are standards issued by the International Functions Point User Group (IFPUG, covering the Albrecht method) and the United Kingdom Function Point User Group (UFGU, covering the MKII method).
- An ISO standard for function point method is also being developed.

EXAMPLE: I

Given the following values, compute function point when all complexity adjustment factor (CAF) and weighting factors are average.

User Input = 50

User Output = 40

User Inquiries = 35

User Files = 6

External Interface = 4

SOLUTION

➤ **Step-I: Calculate the degree of influence(F)**

$$F = 14 * \text{scale}$$

Scale varies from 0 to 5 according to character of Complexity Adjustment Factor (CAF). Below table shows scale:

- 0 - No Influence
- 1 - Incidental
- 2 - Moderate
- 3 - Average
- 4 - Significant
- 5 - Essential

As complexity adjustment factor is average (given in question), hence,

$$\text{scale} = 3.$$

Therefore, $F = 14 * 3 = 42$

- Step -2: Calculate Complexity Adjustment Factor (CAF),

$$\text{CAF} = 0.65 + (0.01 * F)$$

$$\text{CAF} = 0.65 + (0.01 * 42) = 1.07$$

- Step-3: As weighting factors are also average (given in question) hence we will multiply each individual function point to corresponding values in TABLE.

$$\text{UFP} = (50*4) + (40*5) + (35*4) + (6*10) + (4*7) = 628$$

- Step -4: Calculate Function Point.

$$\text{FP} = \text{UFP} * \text{CAF}$$

$$\text{Therefore, Function Point} = 628 * 1.07 = 671.96$$

Function Units	Low	Avg	High
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

EXAMPLE:2

An application has the following:

10 low external inputs, 12 high external outputs, 20 low internal logical files, 15 high external interface files, 12 average external inquiries, and a value of complexity adjustment factor of 1.10.

What are the **unadjusted and adjusted function point counts(FP)**?

SOLUTION

According to question, there is 10 low EI, 12 high EO, 20 low ILF, 15 high EIF and 12 average EQ

Unadjusted function point counts may be calculated using as:

$$\begin{aligned} &= 10 \times 3 + 12 \times 7 + 20 \times 7 + 15 \times 10 + 12 \times 4 \\ &= 30 + 84 + 140 + 150 + 48 \\ &= 452 \end{aligned}$$

$$\begin{aligned} \text{FP} &= \text{UFP} \times \text{CAF} \\ &= 452 \times 1.10 = 497.2 \end{aligned}$$

EXAMPLE: 3

Consider a project with these parameters:

- (i) External Inputs:
 - (a) 10 with low complexity
 - (b) 15 with average complexity
 - (c) 17 with high complexity
- (ii) External Outputs:
 - (a) 6 with low complexity
 - (b) 13 with high complexity
- (iii) External Inquiries:
 - (a) 3 with low complexity
 - (b) 4 with average complexity
 - (c) 2 high complexity

(iv) Internal logical files:

(a)2 with average complexity

(b)1 with high complexity

(v) External Interface files:

(a)9 with low complexity

In addition to above, system requires

i. Significant data communication

ii. Performance is very critical

iii. Designed code may be moderately reusable

iv. System is not designed for multiple installation in different organizations.

Other complexity adjustment factors are treated as average.

Compute the function points for the project.

Solution: Unadjusted function points may be counted using table 2

Functional Units	Count	Complexity		Complexity Totals	Functional Unit Totals
External Inputs (EIs)	10	Low x 3	=	30	192
	15	Average x 4	=	60	
	17	High x 6	=	102	
External Outputs (EOs)	6	Low x 4	=	24	115
	0	Average x 5	=	0	
	13	High x 7	=	91	
External Inquiries (EQs)	3	Low x 3	=	9	37
	4	Average x 4	=	16	
	2	High x 6	=	12	
External logical Files (ILFs)	0	Low x 7	=	0	35
	2	Average x 10	=	20	
	1	High x 15	=	15	
External Interface Files (EIFs)	9	Low x 5	=	45	45
	0	Average x 7	=	0	
	0	High x 10	=	0	
Total Unadjusted Function Point Count					424

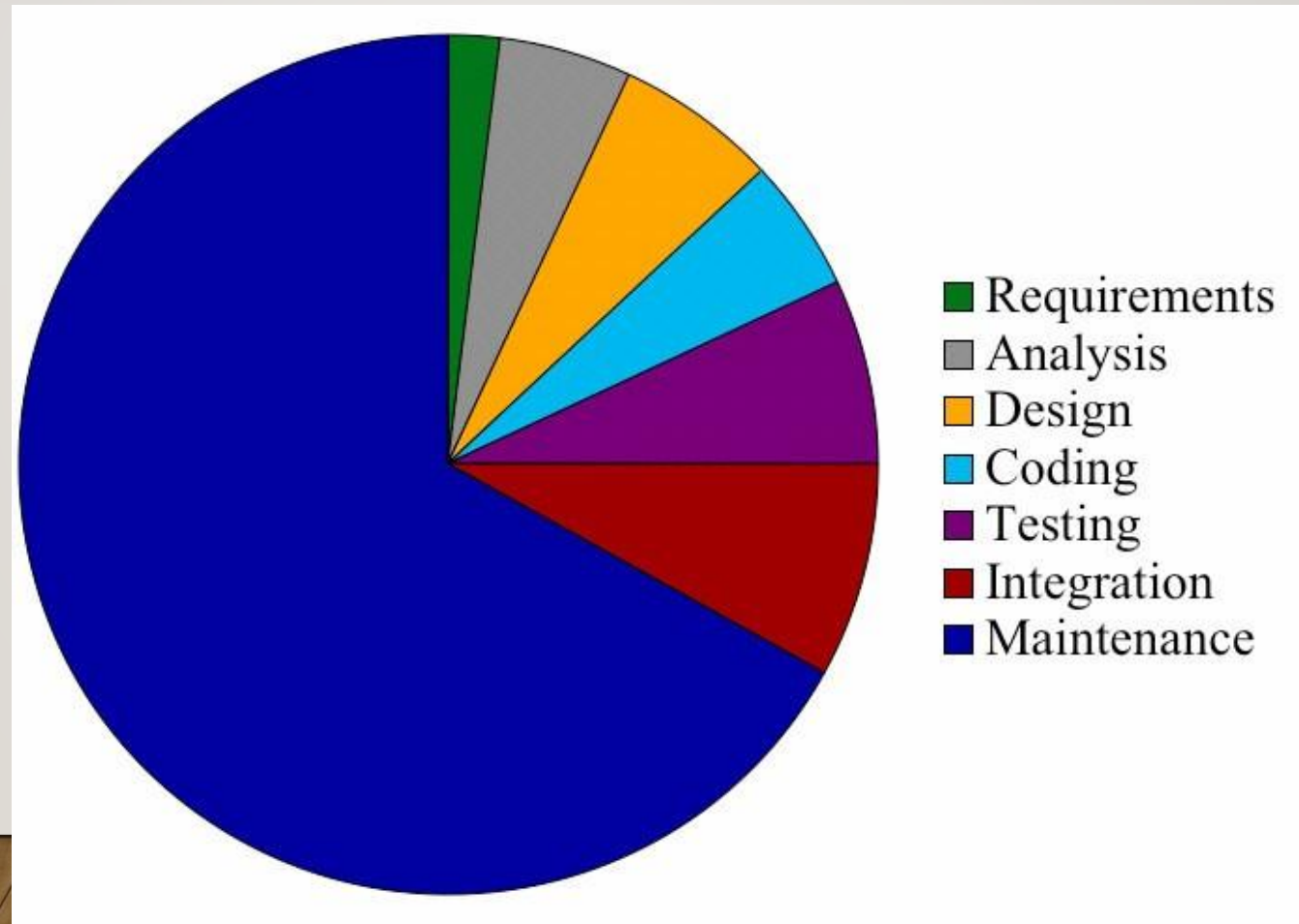
$$\sum_{i=1}^{14} F_i = 3+4+3+5+3+3+3+3+3+3+3+2+3+0+3=41$$

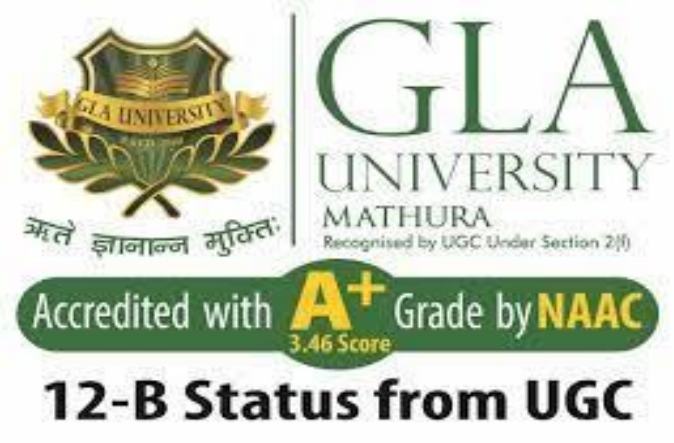
$$\begin{aligned}\text{CAF} &= (0.65 + 0.01 \times \Sigma F_i) \\ &= (0.65 + 0.01 \times 41) \\ &= 1.06\end{aligned}$$

$$\begin{aligned}\text{FP} &= \text{UFP} \times \text{CAF} \\ &= 424 \times 1.06 \\ &= 449.44\end{aligned}$$

Hence $\text{FP} = 449$

RELATIVE COST OF SOFTWARE PHASES





ESTIMATION MODELS

MS. CHESHTAA BHARDWAJ

ASSISTANT PROFESSOR, DEPT. OF CEA,

GLA UNIVERSITY, MATHURA

CONSTRUCTIVE COST MODEL(COCOMO MODEL)

- The COCOMO Model is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as **size, effort, cost, time, and quality**. It was proposed by **Barry Boehm in 1981**.
- The key parameters that define the quality of any software products, which are also an outcome of the COCOMO are primarily Effort and schedule:
 - **--Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.
 - **--Development Time:** This simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, and months.

CHARACTERISTICS OF DIFFERENT PROJECT TYPES

- Organic
- Semi-Detached
- Embedded

Mode	Project size	Nature of Project	Innovation	Deadline of the project	Development Environment
Organic	Typically 2-50 KLOC	Small size project, experienced developers in the familiar environment. For example, pay roll, inventory projects etc.	Little	Not tight	Familiar & In house
Semi detached	Typically 50-300 KLOC	Medium size project, Medium size team, Average previous experience on similar project. For example: Utility systems like compilers, database systems, editors etc.	Medium	Medium	Medium
Embedded	Typically over 300 KLOC	Large project, Real time systems, Complex interfaces, Very little previous experience. For example: ATMs, Air Traffic Control etc.	Significant	Tight	Complex Hardware/ customer Interfaces required

TYPES OF COCOMO MODEL

- Basic
- Intermediate
- Detailed

BASIC- COCOMO MODEL

Basic COCOMO model takes the form

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (E)^{d_b}$$

where E is effort applied in Person-Months, and D is the development time in months. The coefficients a_b , b_b , c_b and d_b are given in table below.

Software Project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Table: Basic COCOMO coefficient

Contd.

When effort and development time are known, the average staff size to complete the project may be calculated as:

$$\text{Average staff size (SS)} = \frac{E}{D} \text{ Persons}$$

When project size is known, the productivity level may be calculated as:

$$\text{Productivity (P)} = \frac{KLOC}{E} KLOC / PM$$

EXAMPLE: 4.5

Suppose that a project was estimated to be **400 KLOC**. Calculate the effort and development time for each of the three modes i.e., organic, semidetached and embedded.

SOLUTION

The basic COCOMO equation take the form:

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (E)^{d_b}$$

Estimated size of the project = 400 KLOC

(i) Organic mode

$$E = 2.4(400)^{1.05} = 1295.31 \text{ PM}$$

$$D = 2.5(1295.31)^{0.38} = 38.07 \text{ M}$$

(ii) Semidetached mode

$$E = 3.0(400)^{1.12} = 2462.79 \text{ PM}$$

$$D = 2.5(2462.79)^{0.35} = 38.45 \text{ M}$$

(iii) Embedded mode

$$E = 3.6(400)^{1.20} = 4772.81 \text{ PM}$$

$$D = 2.5(4772.8)^{0.32} = 38 \text{ M}$$

EXAMPLE: 4.6

A project size of **200 KLOC** is to be developed.

Software development team has average experience on similar type of projects. The project schedule is not very tight. Calculate the effort, development time, average staff size and productivity of the project.

The **semi-detached mode** is the most appropriate mode; keeping in view the size, schedule and experience of the development team.

$$\text{Hence } E = 3.0(200)^{1.12} = 1133.12 \text{ PM}$$

$$D = 2.5(1133.12)^{0.35} = 29.3 \text{ M}$$

$$\text{Average staff size } (SS) = \frac{E}{D} \text{ Persons}$$

$$= \frac{1133.12}{29.3} = 38.67 \text{ Persons}$$

$$\text{Productivity} = \frac{KLOC}{E} = \frac{200}{1133.12} = 0.1765 KLOC / PM$$

$$P = 176 LOC / PM$$

INTERMEDIATE MODEL

Cost drivers

(i) Product Attributes

- Required s/w reliability
- Size of application database
- Complexity of the product

(ii) Hardware Attributes

- Run time performance constraints
- Memory constraints
- Virtual machine volatility
- Turnaround time

Contd.

(iii) Personal Attributes

- Analyst capability
- Programmer capability
- Application experience
- Virtual m/c experience
- Programming language experience

(iv) Project Attributes

- Modern programming practices
- Use of software tools
- Required development Schedule

Multipliers for Effort Adjustment Factor (EAF) of different cost drivers

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Product Attributes						
RELY	0.75	0.88	1.00	1.15	1.40	--
DATA	--	0.94	1.00	1.08	1.16	--
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
TIME	--	--	1.00	1.11	1.30	1.66
STOR	--	--	1.00	1.06	1.21	1.56
VIRT	--	0.87	1.00	1.15	1.30	--
TURN	--	0.87	1.00	1.07	1.15	--

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Ver y hig h	Extr a hig h
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	--
AEXP	1.29	1.13	1.00	0.91	0.82	--
PCAP	1.42	1.17	1.00	0.86	0.70	--
VEXP	1.21	1.10	1.00	0.90	--	--
LEXP	1.14	1.07	1.00	0.95	--	--
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	--
TOOL	1.24	1.10	1.00	0.91	0.83	--
SCED	1.23	1.08	1.00	1.04	1.10	--

Intermediate COCOMO equations

$$E = a_i (KLOC)^{b_i} \times EAF$$

$$D = c_i (E)^{d_i}$$

Note: **EAF** is **Effort Adjustment Factor** calculated by multiplying the value of different cost drivers with their respective ratings (using table shown on previous two slides)

Project	a_i	b_i	c_i	d_i
Organic	3.2	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Table: Coefficients for intermediate COCOMO

EXAMPLE: 4.7

A new project with estimated **400 KLOC embeddedsystem** has to be developed.

Project manager has a choice of hiring from two pools of developers:

Very highly capable with very little experience in the programming language being used.

Or

Developers of low quality but a lot of experience with the programming language.

What is the impact of hiring all developers from one or the other pool ?



SOLUTION

This is the case of embedded mode and model is intermediate COCOMO.

Hence

$$E = a_i (KLOC)^{b_i} \times EAF$$
$$= 2.8 (400)^{1.20} \times EAF = 3712 \text{ PM}$$

Case I: Developers are very highly capable with very little experience in the programming being used.

$$\begin{aligned} EAF &= 0.82 \times 1.14 = 0.9348 \\ E &= 3712 \times .9348 = 3470 \text{ PM} \\ D &= 2.5 (3470)^{0.32} = 33.9 \text{ M} \end{aligned}$$

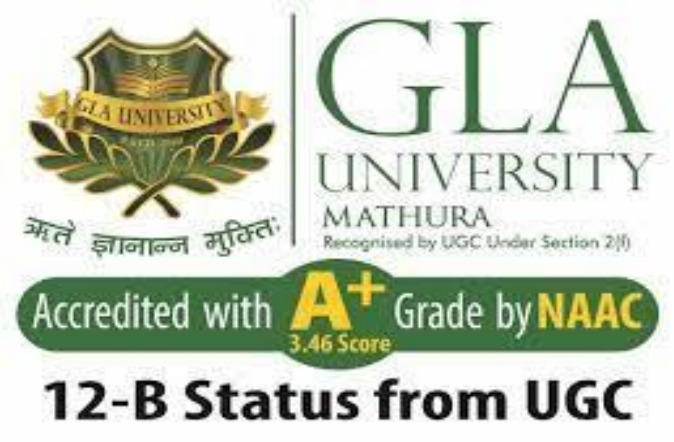
Highly Capable means AEXP = 0.82
&
Very Little Experience means LEXP = 1.14

Case II: Developers are of low quality but lot of experience with the programming language being used.

$$\begin{aligned} \text{EAF} &= 1.29 \times 0.95 = 1.22 \\ \text{E} &= 3712 \times 1.22 = 4528 \text{ PM} \\ \text{D} &= 2.5 (4528)^{0.32} = 36.9 \text{ M} \end{aligned}$$

Low Quality means AEXP = 1.29
&
Lot of Experience (high) means LEXP = 0.95

Case II requires more effort and time. Hence, low quality developers with lot of programming language experience could not match with the performance of very highly capable developers with very little experience.



SOFTWARE DESIGN

MS. CHESHTAA BHARDWAJ

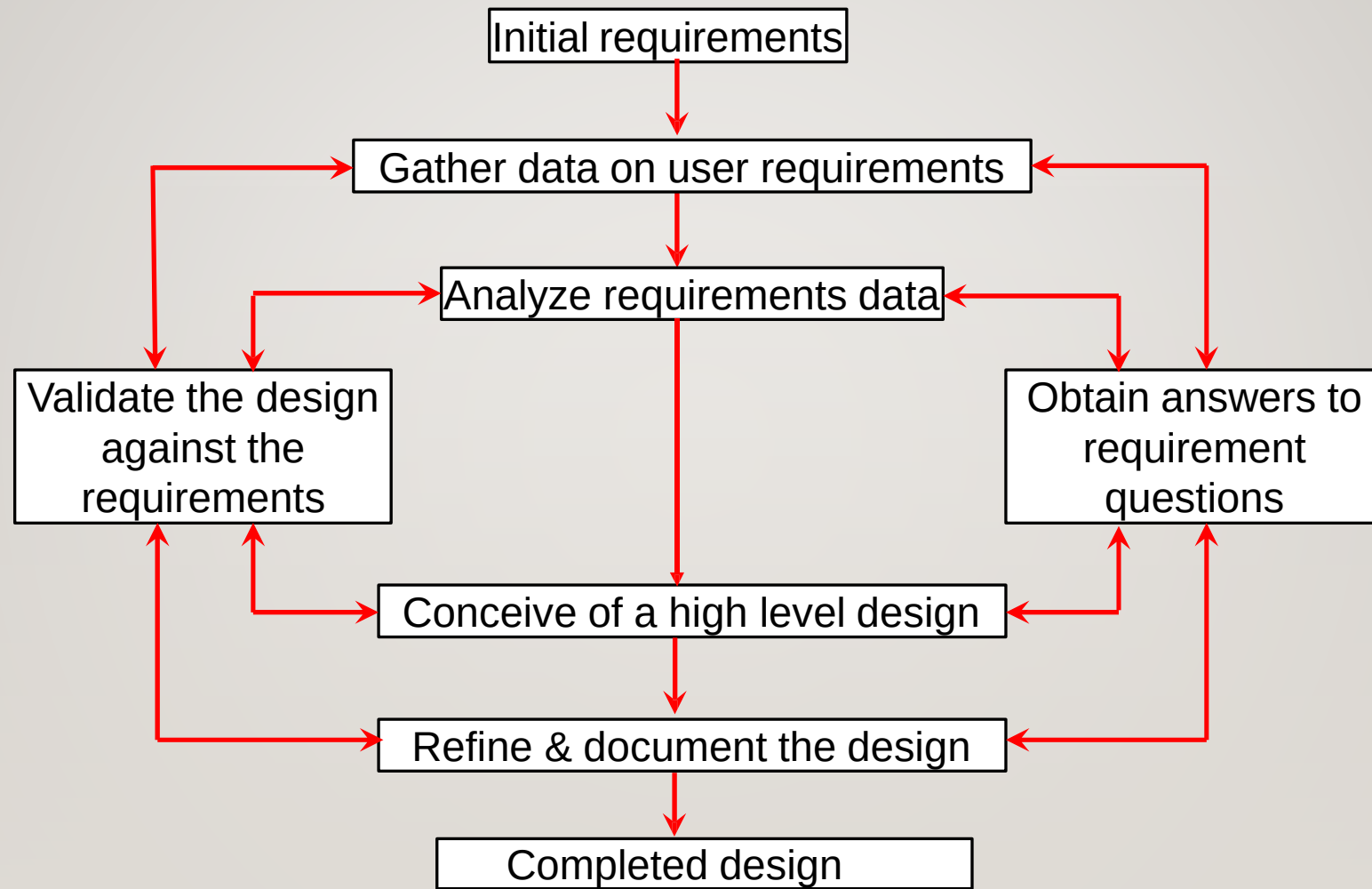
ASSISTANT PROFESSOR, DEPT. OF CEA,

GLA UNIVERSITY, MATHURA

INTRODUCTION

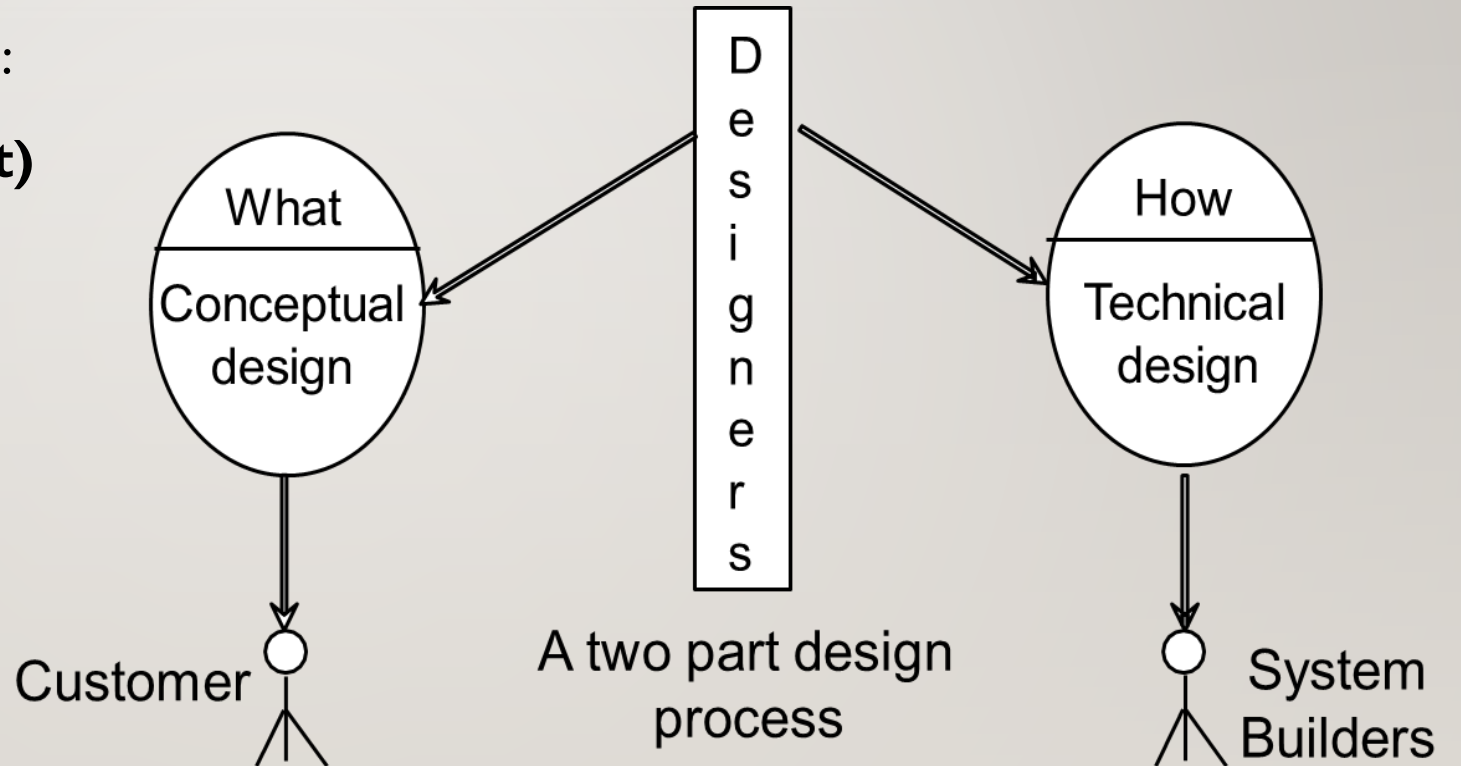
- Software Design is a mechanism to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
- It deals with representing the client's requirement, as described in SRS (Software Requirement Specification) document, into a form, i.e., easily implementable using programming language.
- It produces a solution to a problem given in the SRS document.
- **The output of the design phase is a Software Design Document (SDD).**

Fig. I : Design framework



DESIGN SATISFY BOTH CUSTOMERS & DEVELOPERS

- Design process has two parts:
 - **Conceptual Design (What)**
 - **Technical Design (How)**



CONCEPTUAL DESIGN ANSWERS

- Where will the data come from ?
- What will happen to data in the system?
- How will the system look to users?
- What choices will be offered to users?
- What is the timings of events?
- How will the reports & screens look like?

TECHNICAL DESIGN DESCRIBES

- Hardware configuration
- Software needs
- Communication interfaces
- I/O of the system
- Software architecture
- Network architecture
- Any other thing that translates the requirements in to a solution to the customer's problem.

OBJECTIVES OF SYSTEM DESIGN

- **Correctness:** Software design should be correct as per requirement.
- **Completeness:** The design should have all components like data structures, modules, and external interfaces, etc.
- **Efficiency:** Resources should be used efficiently by the program.
- **Flexibility:** Able to modify on changing needs.
- **Consistency:** There should not be any inconsistency in the design.
- **Maintainability:** The design should be so simple so that it can be easily maintainable by other designers.

MODULARITY

- **Modularization is the process of dividing a software system into multiple independent modules where each module works independently.**
- There are many advantages of Modularization in software engineering. Some of these are given below:
 - Easy to understand the system.
 - System maintenance is easy.
 - A module can be used many times as their requirements. No need to write it again and again.
- A modular system consist of well-defined manageable units with well defined interfaces among the units.

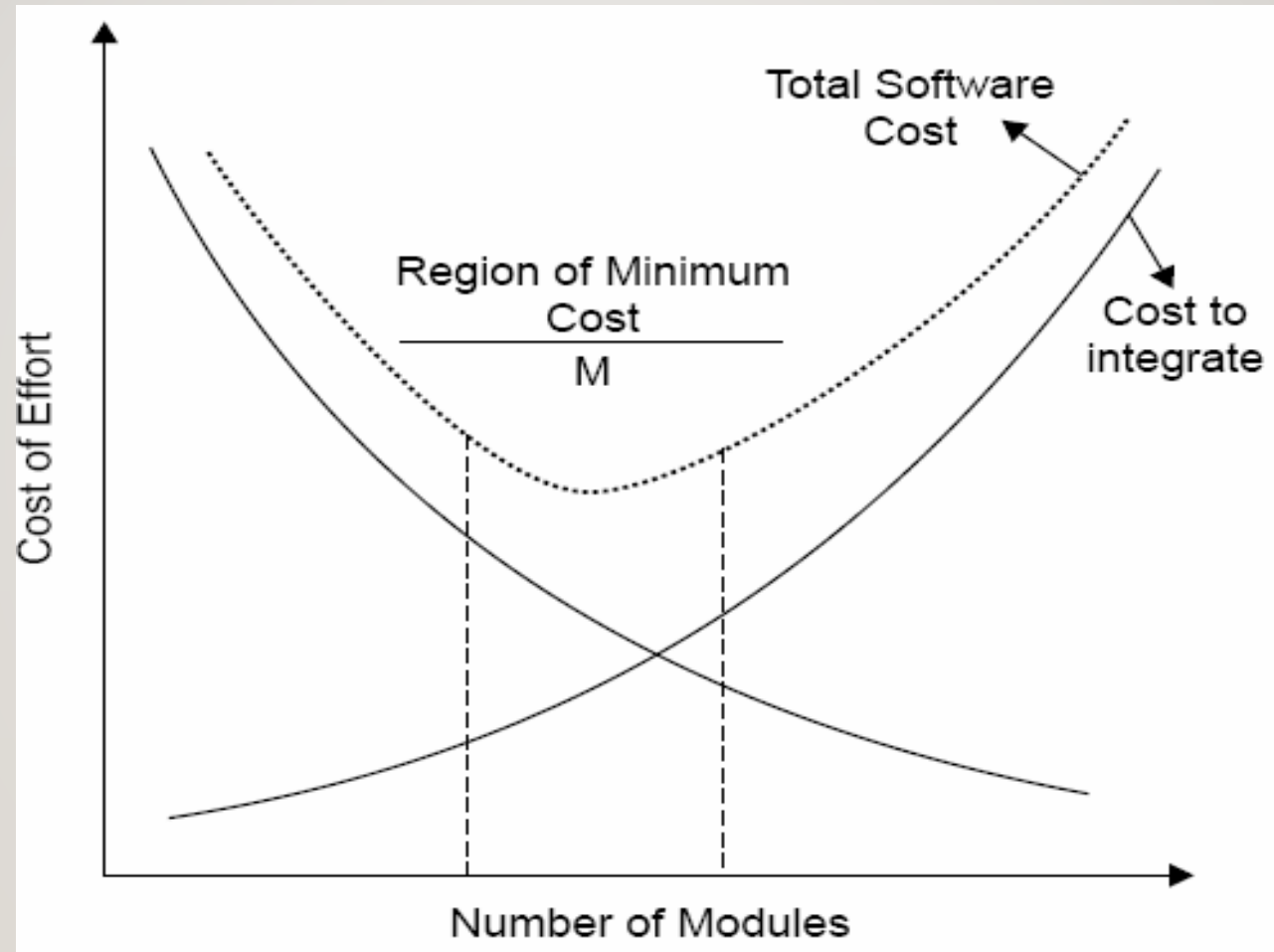
Contd.

- Modularity is the single attribute of software that allows a program to be intellectually manageable.
- It enhances design clarity, which in turn eases implementation, debugging, testing, documenting and maintenance of the software product.

PROPERTIES OF MODULARIZATION

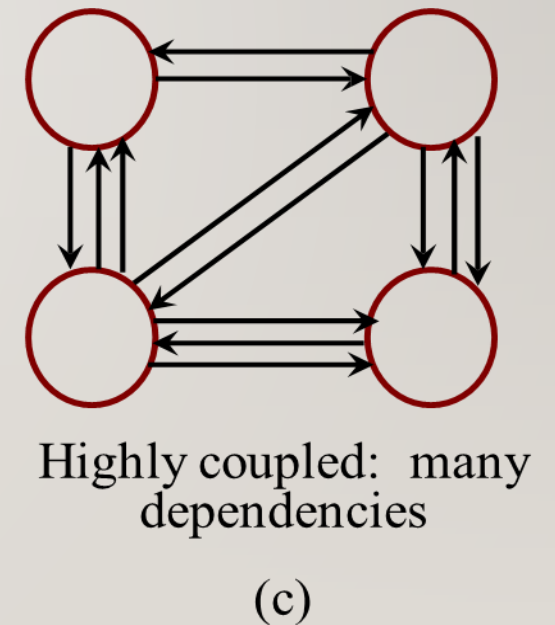
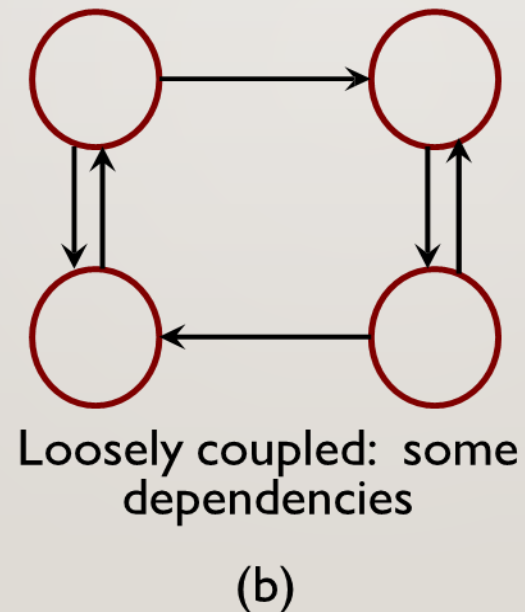
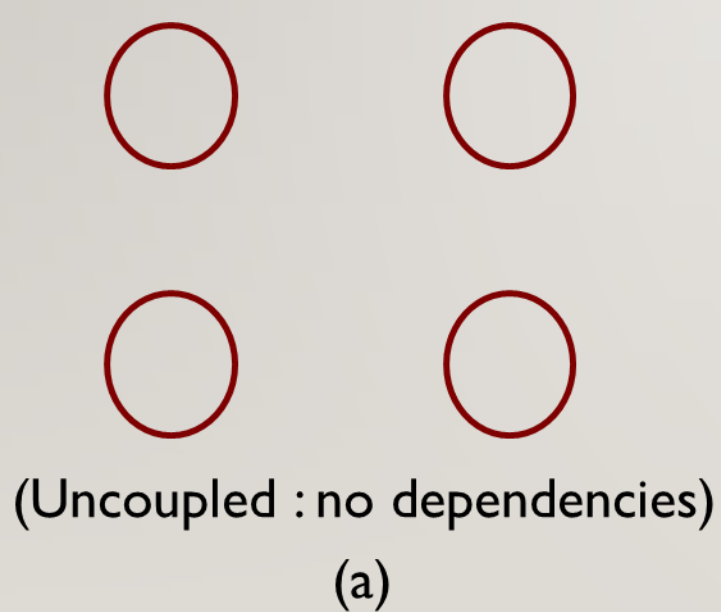
- Well defined subsystem
- Well defined purpose
- Can be separately compiled and stored in a library.
- Module can use other modules
- Module should be easier to use than to build
- Simpler from outside than from the inside.

Modularity and software cost



COUPLING

- Coupling is the measure of the degree of interdependence between the modules.
- **A good software will have low coupling.**

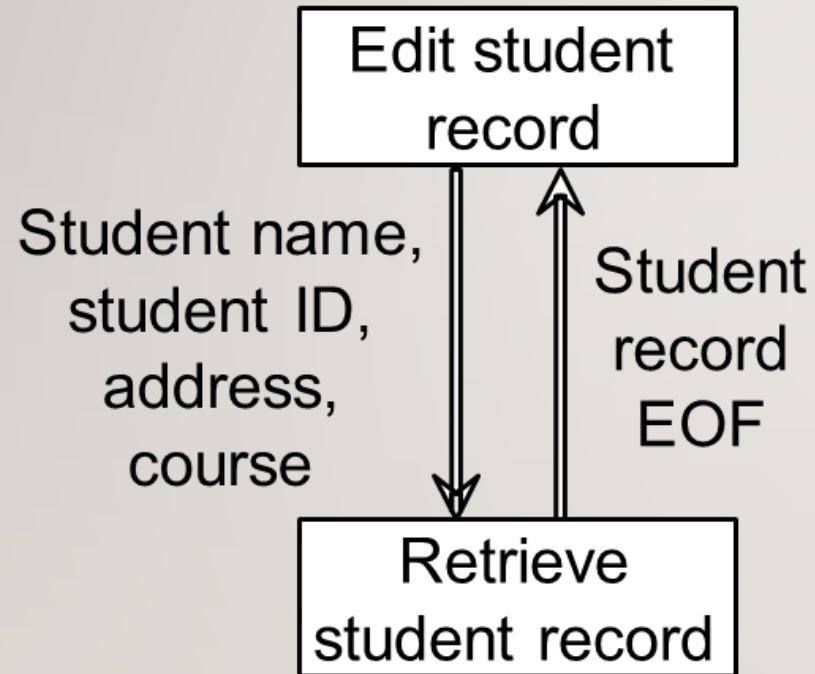


THIS CAN BE ACHIEVED BY

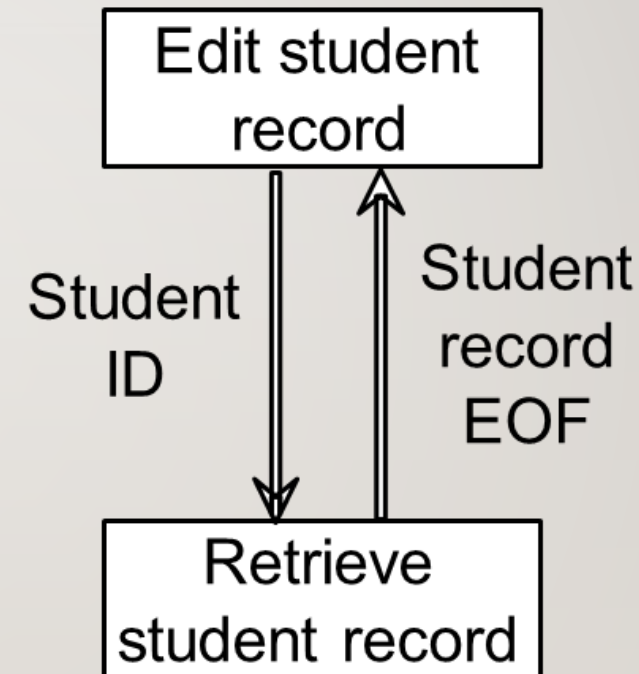
- Controlling the number of parameters passed amongst modules.
- Avoid passing undesired data to calling module.
- Maintain parent/child relationship between calling & called modules.
- Pass data, not the control information.

Example of coupling

Consider the example of editing a student record in a 'student information system'.



Poor design: Tight Coupling



Good design: Loose Coupling

TYPES OF COUPLING

Data coupling	Best
Stamp coupling	
Control coupling	
External coupling	
Common coupling	
Content coupling	Worst

DATA COUPLING

- If the dependency between the modules is based on the fact that they communicate by passing only data, then the modules are said to be data coupled.
- In data coupling, the components are independent of each other and communicate through data.

STAMP COUPLING

- In stamp coupling, the complete data structure is passed from one module to another module.

CONTROL COUPLING

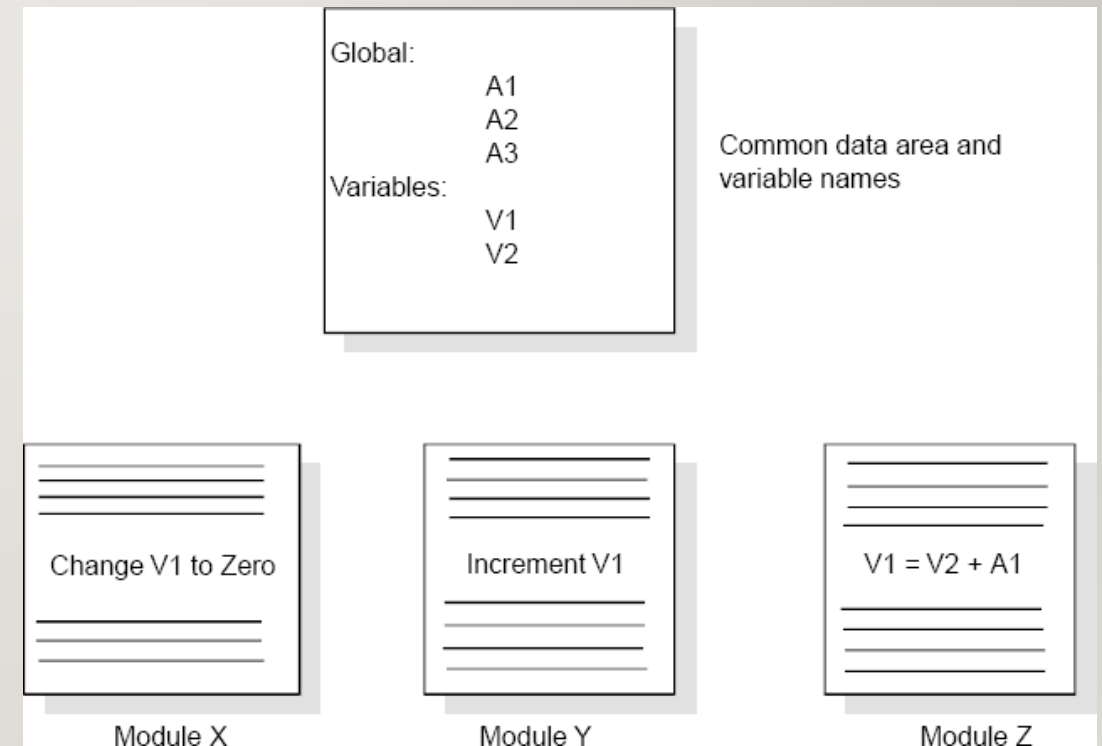
- If the modules communicate by passing control information, then they are said to be control coupled.
- It can be bad if parameters indicate completely different behavior and good if parameters allow factoring and reuse of functionality.

EXTERNAL COUPLING

- In external coupling, the modules depend on other modules, external to the software being developed or to a particular type of hardware.
- Ex- protocol, external file, device format, etc.

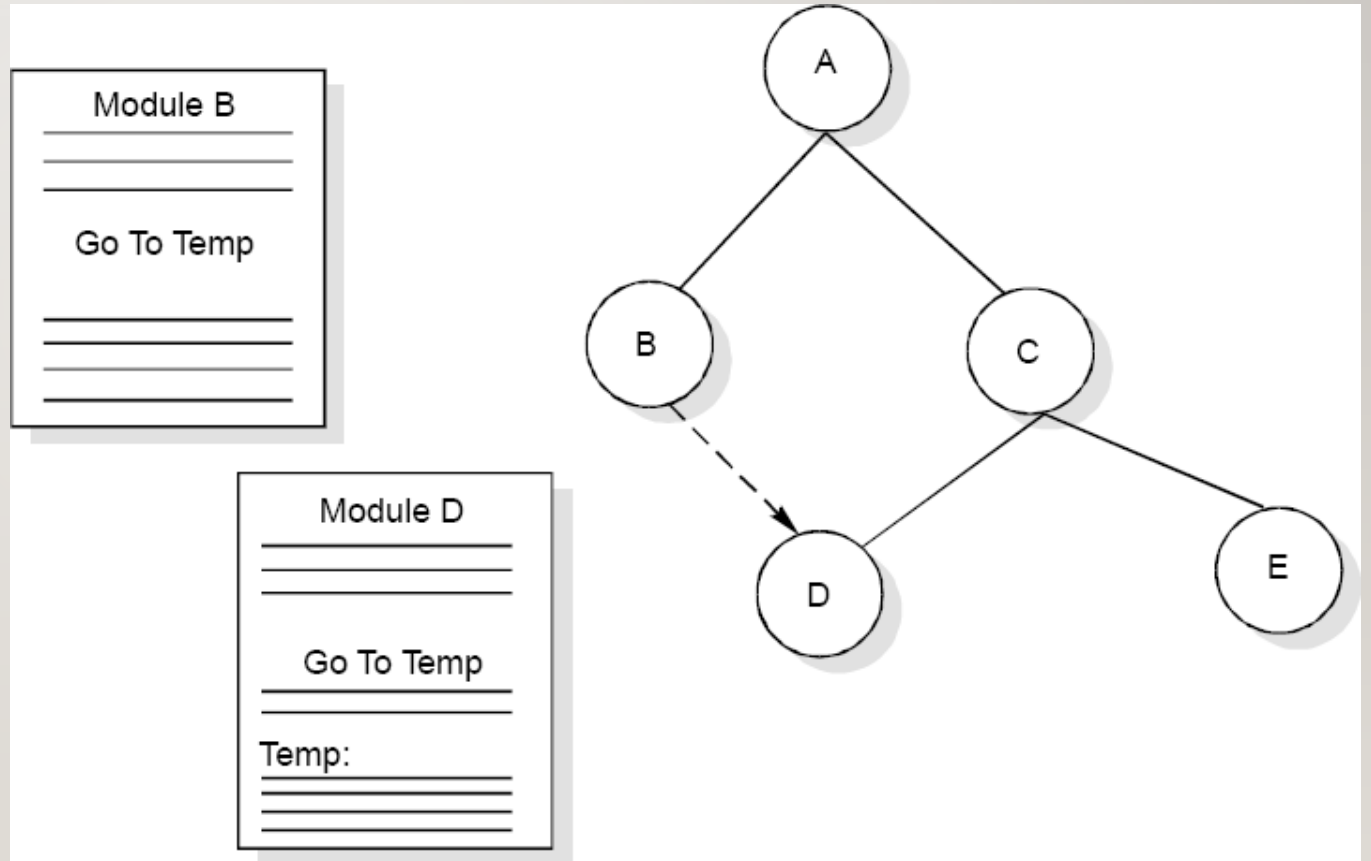
COMMON COUPLING

- **The modules have shared data such as global data structures.** The changes in global data mean tracing back to all modules which access that data to evaluate the effect of the change.
- So it has got disadvantages like difficulty in reusing modules, reduced ability to control data accesses, and reduced maintainability.



CONTENT COUPLING

- In a content coupling, one module can modify the data of another module, or control flow is passed from one module to the other module.
- This is the worst form of coupling and should be avoided.



COHESION

- Cohesion is a measure of the **degree to which the elements of the module are functionally related.**
- It is the degree to which all elements directed towards performing a single task are contained in the component.
- Basically, cohesion is the internal glue that keeps the module together.
- **A good software design will have high cohesion.**

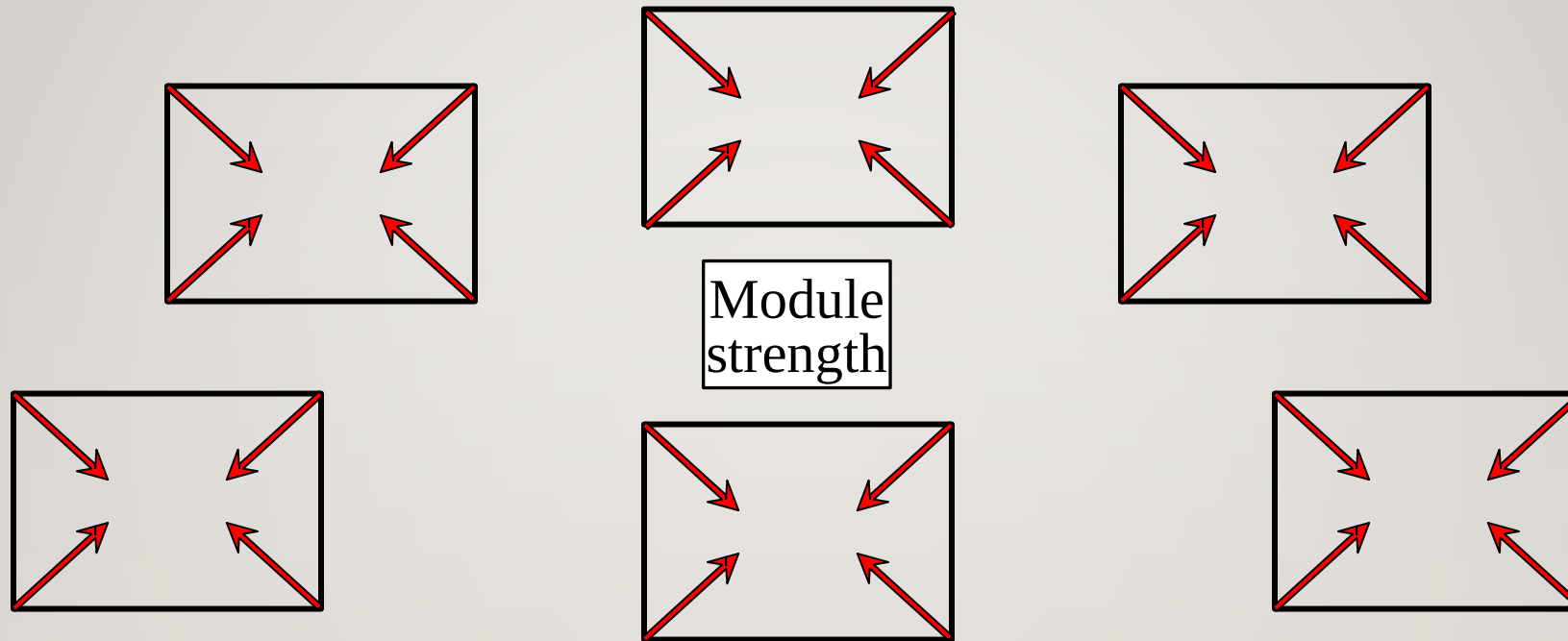


Fig: Cohesion=Strength of relations within modules

TYPES OF COHESION

Functional Cohesion	Best (high)	
Sequential Cohesion		
Communicational Cohesion		
Procedural Cohesion		
Temporal Cohesion		
Logical Cohesion		
Coincidental Cohesion	Worst (low)	

FUNCTIONAL COHESION

- Every essential element for a single computation is contained in the component.
- A and B are part of a single functional task. This is very good reason for them to be contained in the same procedure.
- A functional cohesion performs the task and functions.
- It is an ideal situation.

SEQUENTIAL COHESION

- An element outputs some data that becomes the input for other element, i.e., data flow between the parts.
- It occurs naturally in functional programming languages.

COMMUNICATIONAL COHESION

- Two elements operate on the same input data or contribute towards the same output data.
- Example- update record in the database and send it to the printer.

PROCEDURAL COHESION

- Procedural cohesion occurs in modules whose instructions although accomplish different task yet have been combined because there is a specific order in which the task are to be completed.
- **Elements of procedural cohesion ensure the order of execution.**
- Ex- calculate student GPA, print student record, calculate cumulative GPA, print cumulative GPA.

TEMPORAL COHESION

- The elements are related by their timing involved.
- A module connected with temporal cohesion all the tasks must be executed in the same time span.
- This cohesion contains the code for initializing all the parts of the system. Lots of different activities occur, all at unit time.

LOGICAL COHESION

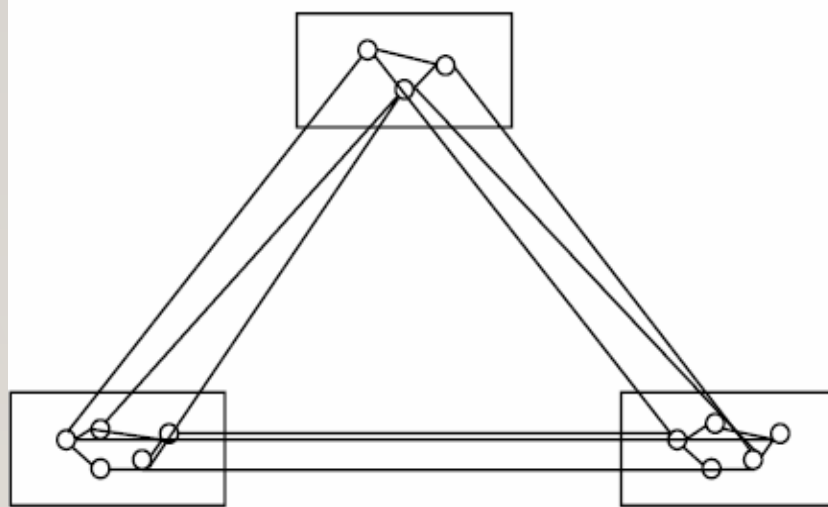
- The elements are logically related and not functionally.
- Ex- A component reads inputs from tape, disk, and network.
- All the code for these functions is in the same component.
- Operations are related, but the functions are significantly different.

COINCIDENTAL COHESION

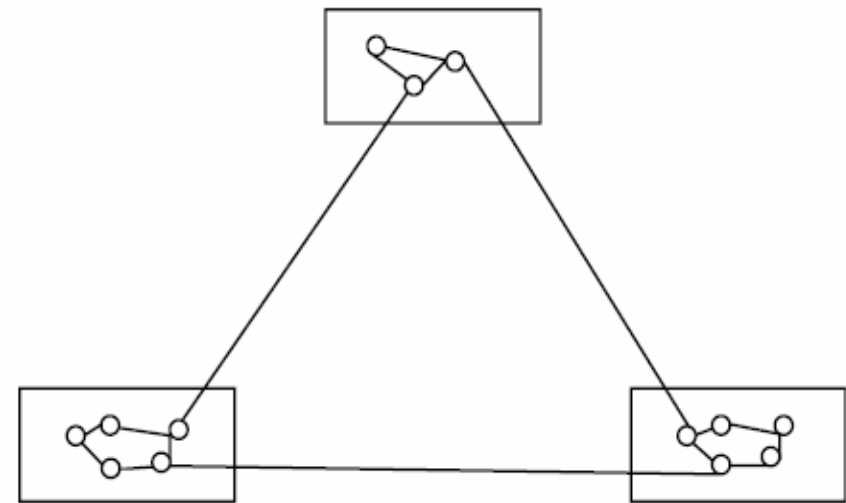
- The elements are not related(unrelated).
- The elements have no conceptual relationship other than location in source code. It is accidental and the worst form of cohesion.
- Ex- print next line and reverse the characters of a string in a single component.

RELATIONSHIP BETWEEN COHESION & COUPLING

- If software isn't modularized effectively, even small enhancements or changes can lead to project failure. Therefore, software engineers should design modules with the aim of achieving **high cohesion and low coupling**.



High Coupling



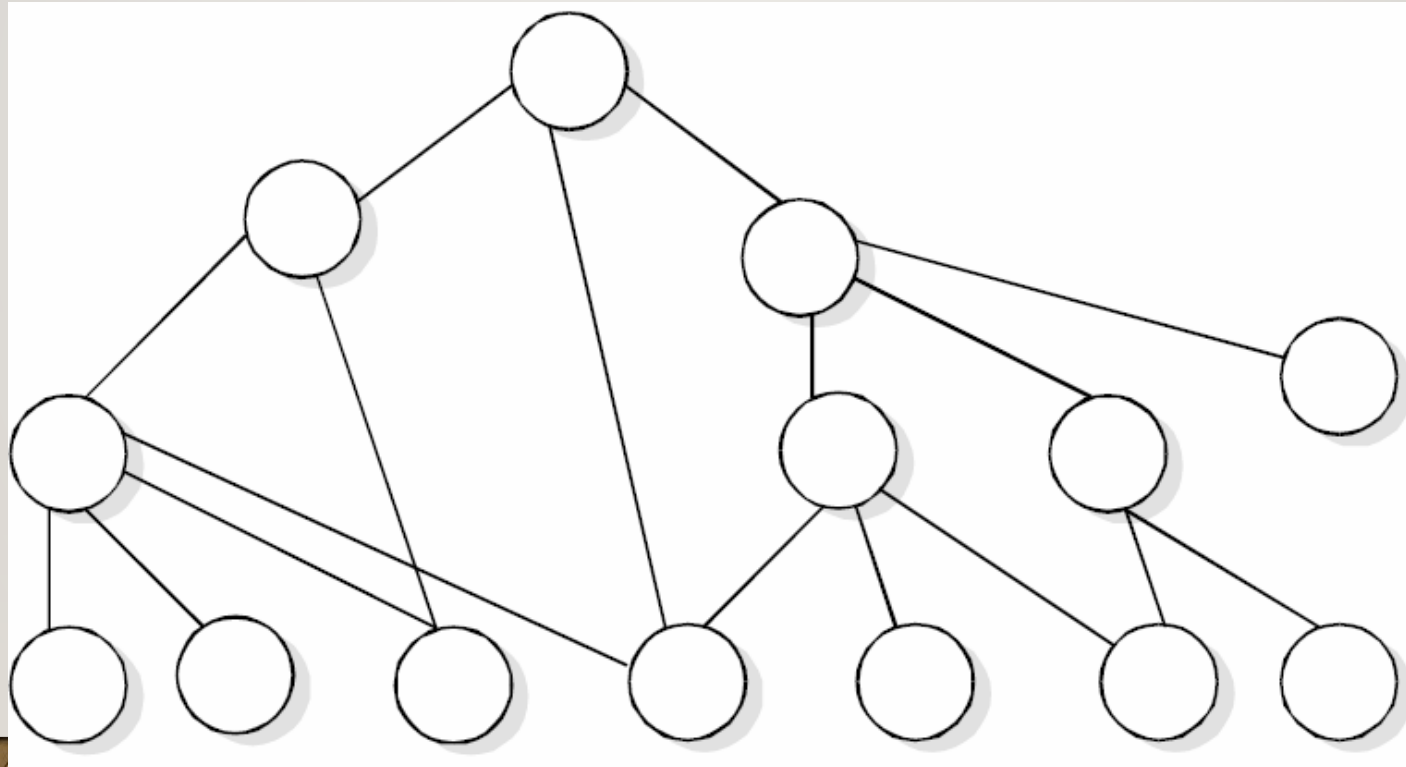
Low Coupling

STRATEGY OF DESIGN

- **A good system design strategy is to organize the program modules in such a way that are easy to develop and latter to change.**
- Structured design techniques help developers to deal with the size and complexity of programs.
- Analysts create instructions for the developers about how code should be written and how pieces of code should fit together to form a program.
- It is important for two reasons:
 - First, even pre-existing code, if any, needs to be understood, organized and pieced together.
 - Second, it is still common for the project team to have to write some code and produce original programs that support the application logic of the system.

BOTTOM-UP DESIGN

- These modules are collected together in the form of a “library”.



TOP-DOWN DESIGN

- A top-down design approach starts by identifying the major modules of the system, decomposing them into their lower-level modules and iterating until the desired level of detail is achieved.
- This is stepwise refinement; starting from an abstract design, in each step the design is refined to a more concrete level, until we reach a level where no more refinement is needed and the design can be implemented directly.

HYBRID DESIGN

- For top-down approach to be effective, some bottom-up approach is essential for the following reasons:
 - To permit common sub modules.
 - Near the bottom of the hierarchy, where the intuition is simpler, and the need for bottom-up testing is greater, because there are more number of modules at low levels than high levels.
 - In the use of pre-written library modules, in particular, reuse of modules.

FUNCTION ORIENTED DESIGN

- The design process for software systems often has two levels. At the first level, the focus is on deciding which modules are needed for the system based on SRS (Software Requirement Specification) and how the modules should be interconnected.
- **Function Oriented Design is an approach to software design where the design is decomposed into a set of interacting units where each unit has a clearly defined function.**

Example:

Consider the example of scheme interpreter. Top-level function may look like:

While (not finished)

```
{  
    Read an expression from the terminal;  
    Evaluate the expression;  
    Print the value;  
}
```

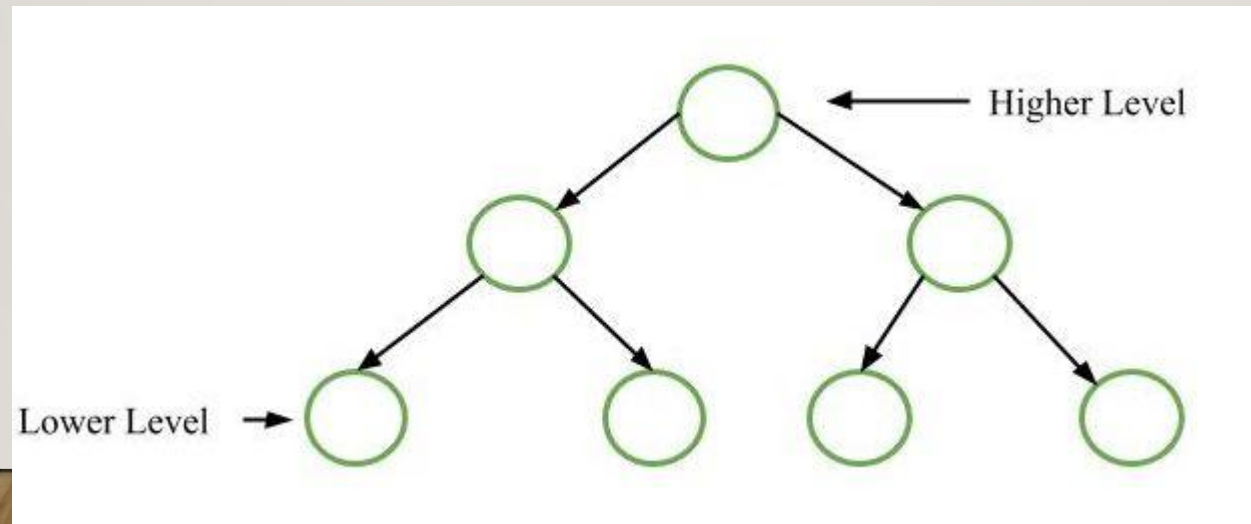
We thus get a fairly natural division of our interpreter into a “read” module, an “evaluate” module and a “print” module. Now we consider the “print” module and is given below:

Print (expression exp)

```
{  
    Switch (exp  $\rightarrow$  type)  
    Case integer: /*print an integer*/  
    Case real:   /*print a real*/  
    Case list:   /*print a list*/  
    ...  
}
```

GENERIC PROCEDURE

- Start with a high-level description of what the software/program does.
- Refine each part of the description by specifying in greater detail the functionality of each part.
- These points lead to a Top-Down Structure.



FUNCTION ORIENTED DESIGN NOTATIONS

- Design notations are largely meant to be used during the process of design and are used to represent design or design decisions.
- For a function-oriented design, the design can be represented graphically or mathematically by the following:
 - **Data flow diagrams**
 - **Data Dictionaries**
 - **Structure Charts**
 - **Pseudocode**

DATA FLOW DIAGRAMS(DFD)

- A data flow diagram (DFD) maps out the flow of information for any process or system.
- It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination.

DATA DICTIONARIES

- Data dictionaries are simply repositories to store information about all data items defined in DFDs.
- At the requirement stage, data dictionaries contains data items.
- Data dictionaries include Name of the item, Aliases (Other names for items), Description / purpose, Related data items, Range of values, Data structure definition / form.

STRUCTURE CHARTS

- Structure chart is the hierarchical representation of system which partitions the system into black boxes (functionality is known to users, but inner details are unknown).
- Components are read from top to bottom and left to right.
- When a module calls another, it views the called module as a black box, passing required parameters and receiving results.
- Structure chart breaks down the entire system into the lowest functional modules and describes the functions and sub-functions of each module of a system in greater detail.

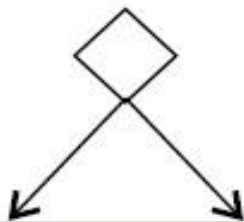
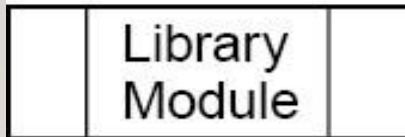
SYMBOLS IN STRUCTURE CHARTS

○→ Data

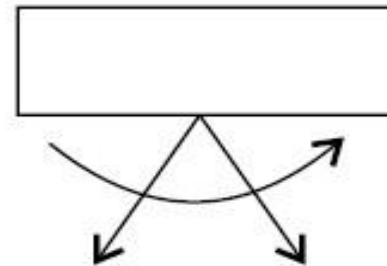
●→ Control

Module

Physical Storage

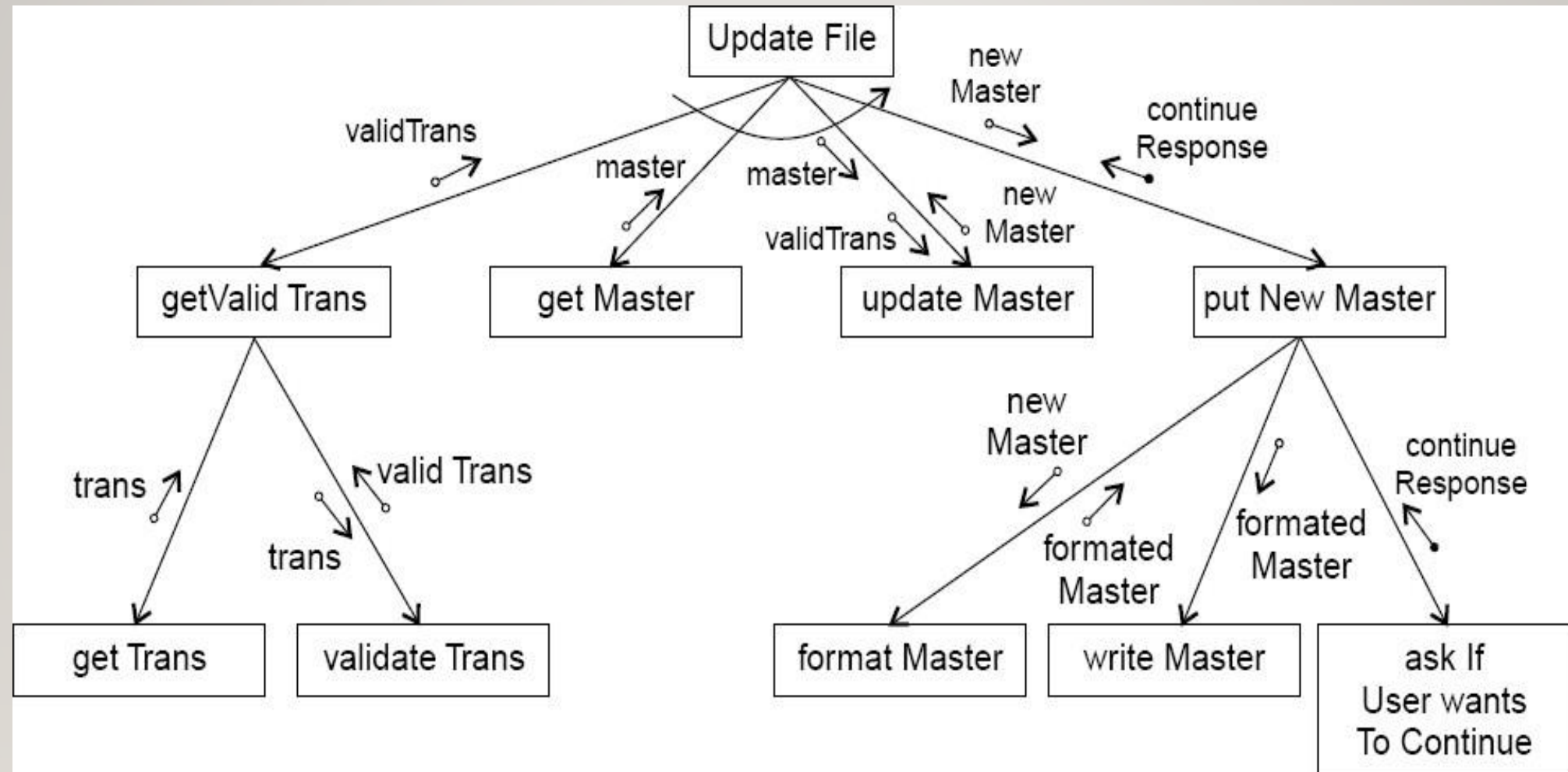


Diamond symbol
for conditional call
of module

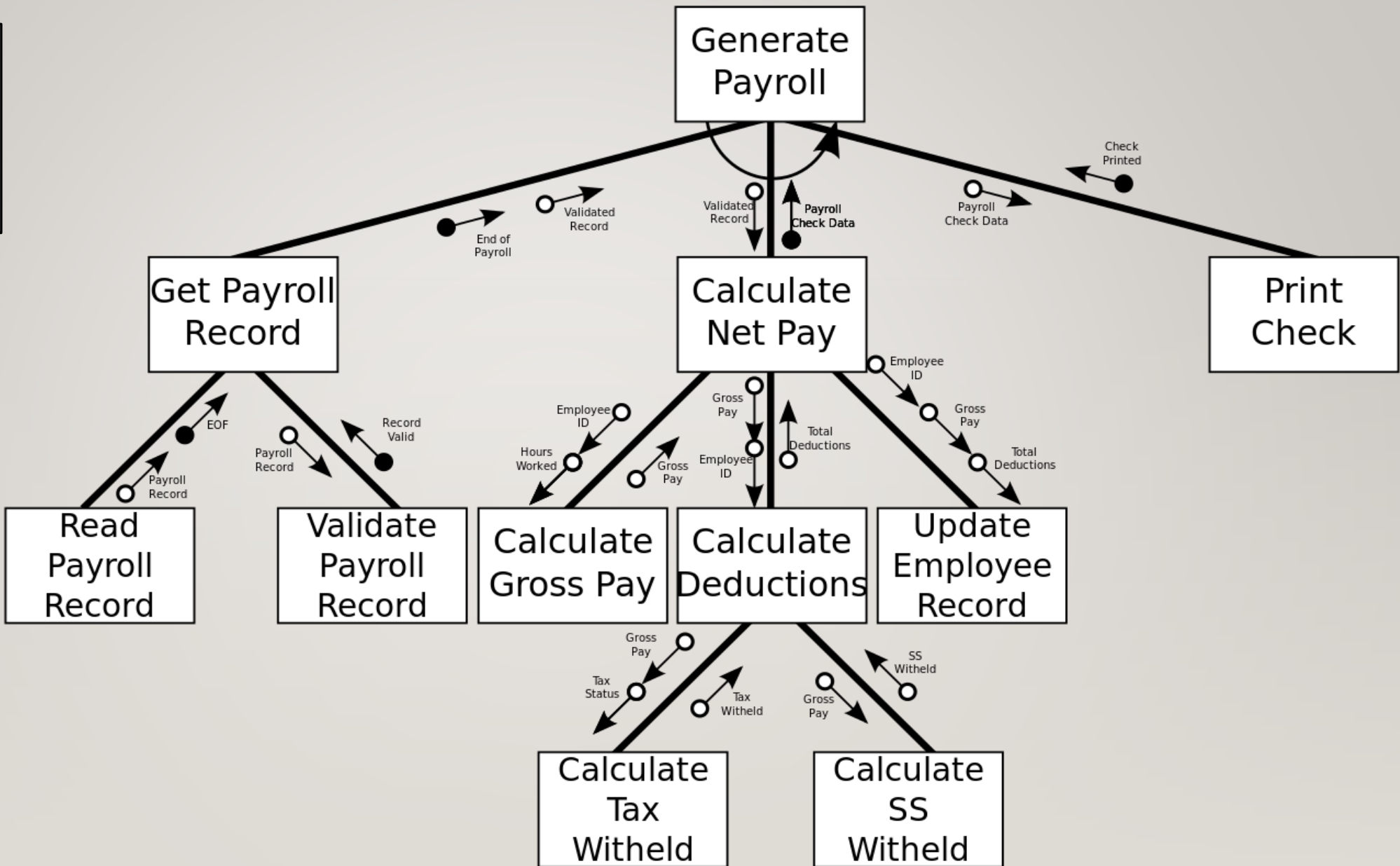


Repetitive call
of module

EXAMPLE FOR “UPDATE FILE”



EXAMPLE
FOR
“GENERATE
PAYROLL”



PSEUDO CODE

- Pseudo Code is system description in short English like phrases describing the function. It uses keyword and indentation.
- Pseudocodes are used as replacement for flow charts. It decreases the amount of documentation required.
- **A Pseudocode is defined as a step-by-step description of an algorithm. Pseudocode does not use any programming language in its representation instead it uses the simple English language text such as It-Then-Else, While-Do, and End, as it is intended for human understanding rather than machine reading.**
- **Pseudocode is the intermediate state between an idea and its implementation(code) in a high-level language**

Pseudocode Example

Good

```
FOR each index of string
  IF string(index) is equal to hash THEN
    RETURN "Hash found"
  END IF
END FOR
```

Bad

```
FOR i = 0 to 20
  IF string[i] = '#'
    RETURN "Hash found"
  END IF
END FOR
```

```
FOR each index of string
  if string[i] = '#'
    return "Hash found"
  END if
END FOR
```

OBJECT ORIENTED DESIGN

- This methodology **employs object-oriented principles to model and design complex systems**. It involves analyzing the problem domain, representing it using objects and their interactions, and then designing a modular and scalable solution.
- Object Oriented Design begins with an examination of the real world “things” that are part of the problem to be solved. These things (which we will call objects) are characterized individually in terms of their attributes and behavior.
- It helps create systems that are easier to understand, maintain, and extend by organizing functionality into reusable and interconnected components.

OBJECTS

- The word “Object” is used as an **entity able to save a state (information) and which offers a number of operations (behavior) to either examine or affect this state.**
- An object is characterized by number of operations and a state which remembers the effect of these operations.

MESSAGES

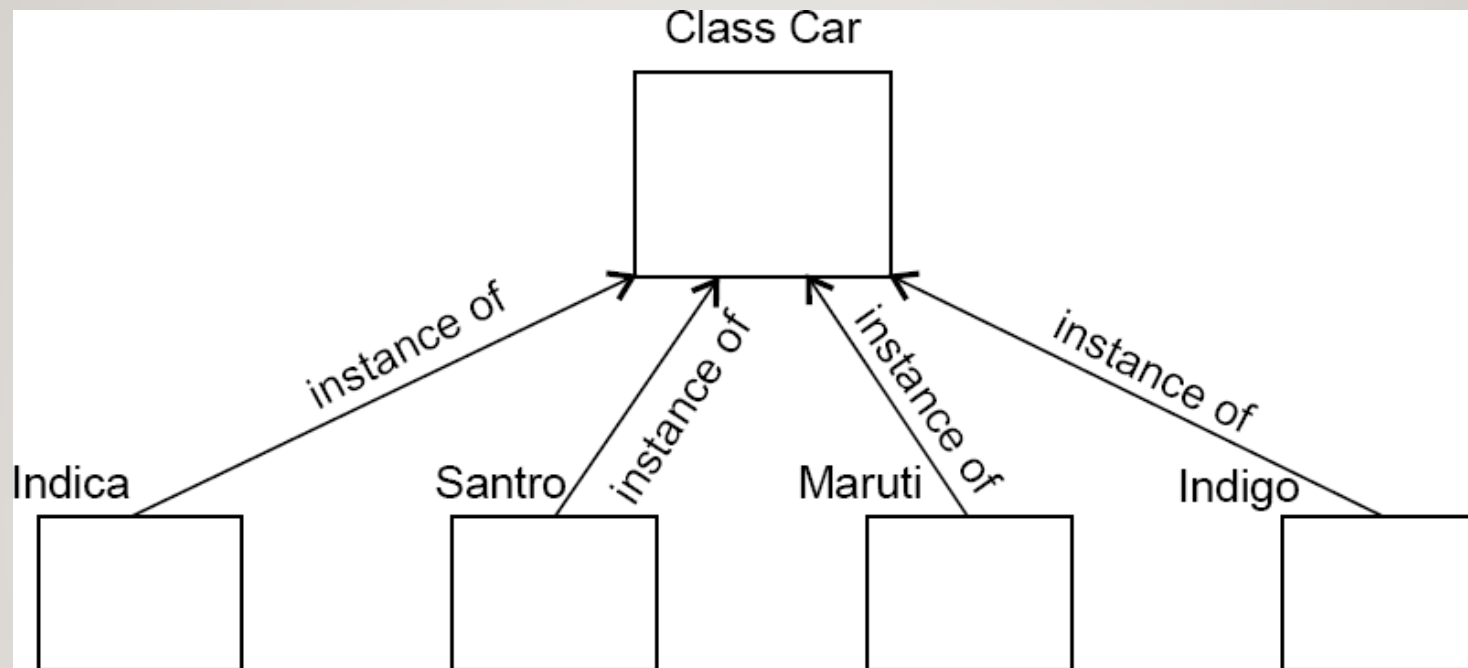
- Objects communicate by message passing.
- **Messages consist of the identity of the target object, the name of the requested operation and any other operation needed to perform the function.**
- Message are often implemented as procedure or function calls.

ABSTRACTION

- In object oriented design, complexity is managed using abstraction.
- **Abstraction is the elimination of the irrelevant and the amplification of the essentials.**

CLASS

- In any system, there shall be number of objects. Some of the objects may have common characteristics and we can group the objects according to these characteristics. This type of grouping is known as a class.
- Hence, **a class is a set of objects that share a common structure and a common behavior.**
- Classes are useful because they act as a blueprint for objects.



Indica, Santro, Maruti, Indigo are all instances of the class “car”

ATTRIBUTES

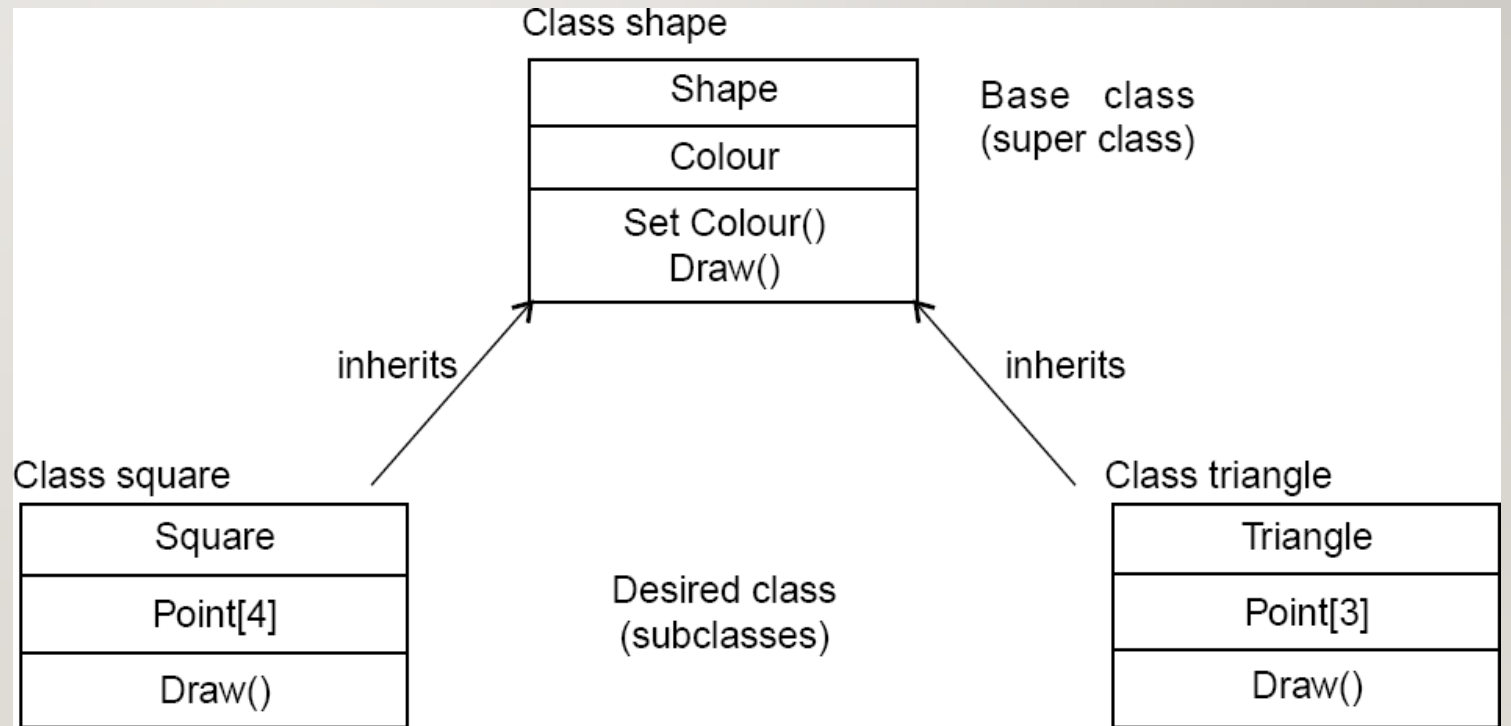
- An attributes is a data value held by the objects in a class.
- Each attributes has a value for each object instance.

OPERATIONS

- **An operation is a function or transformation that may be applied to or by objects in a class.**
- All objects in a class share the same operations.

INHERITANCE

- **Abstracting common features in a new class.**
- The low level classes (known as subclasses or derived classes) inherit state and behavior from this high level class (known as a super class or base class).



POLYMORPHISM

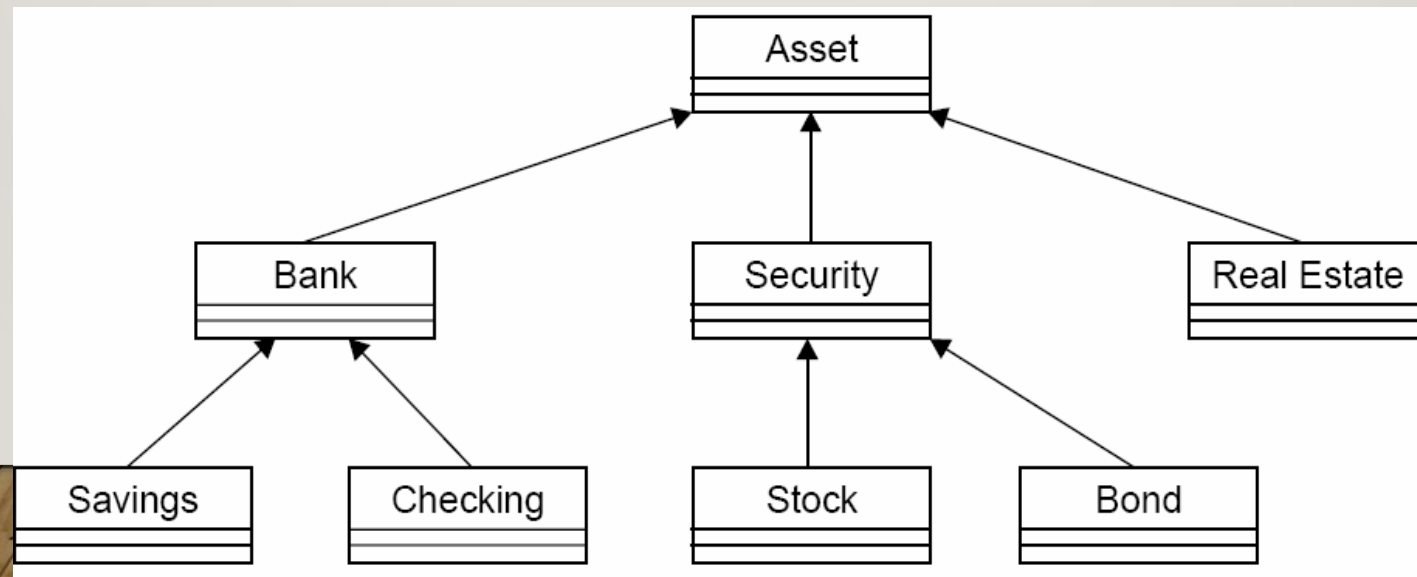
- When we abstract just the interface of an operation and leave the implementation to subclasses it is called a polymorphic operation and process is called polymorphism.

ENCAPSULATION

- Encapsulation is also commonly referred to as “**Information Hiding**”.
- It consists of the separation of the external aspects of an object from the internal implementation details of the object.

HIERARCHY

- Hierarchy involves organizing something according to some particular order or rank.
- It is another mechanism for reducing the complexity of software by being able to treat and express sub-types in a generic way.



SOFTWARE DESIGN DESCRIPTION(SDD)

- An SDD is a representation of a software system that is used as a medium for communicating software design information.
- **The purpose of an SDD is to show how the software system will be structured to satisfy the requirements identified in the SRS.**
- **It is basically the translation of requirements into a description of the software structure, software components, interfaces, and data necessary for the implementation phase.**
- Hence, SDD becomes the blue print for the implementation activity.

- Design Description Information Content
 - Introduction
 - Design entities
 - Design entity attributes
- The attributes and associated information items are defined in the following subsections:
 - Identification
 - Type
 - Purpose
 - Subordinates
 - Dependencies
 - Interface
 - Resources
 - Processing
 - Data

FEW TERMINOLOGIES

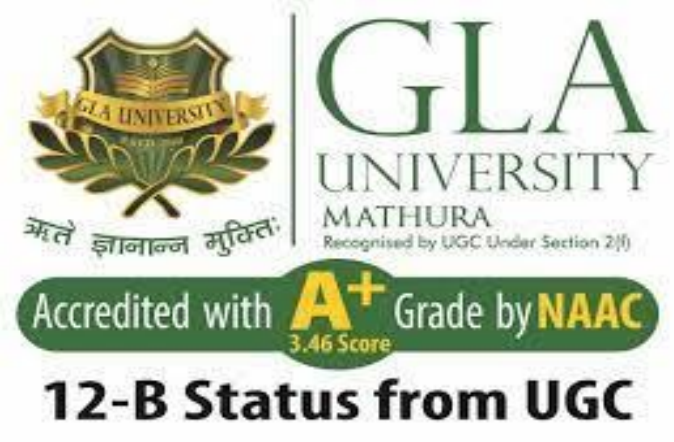
- **Design entity:** An element (Component) of a design that is structurally and functionally distinct from other elements and that is separately named and referenced.
- **Design View:** A subset of design entity attribute information that is specifically suited to the needs of a software project activity.
- **Entity attributes:** A named property or characteristics of a design entity. It provides a statement of fact about the entity.

SOFTWARE DESIGN DESCRIPTION (SDD) ORGANISATION

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions and acronyms
2. References
3. Decomposition description
 - 3.1 Module decomposition
 - 3.1.1 Module 1 description
 - 3.1.2 Module 2 description
 - 3.2 Concurrent Process decomposition
 - 3.2.1 Process 1 description
 - 3.2.2 Process 2 description
 - 3.3 Data decomposition
 - 3.3.1 Data entity 1 description
 - 3.3.2 Data entity 2 description

4. Dependency description
 - 4.1 Intermodule dependencies
 - 4.2 Interprocess dependencies
 - 4.3 Data dependencies
5. Interface description
 - 5.1 Module Interface
 - 5.1.1 Module 1 description
 - 5.1.2 Module 2 description
 - 5.2 Process interface
 - 5.2.1 Process 1 description
 - 5.2.2 Process 2 description
6. Detailed design
 - 6.1 Module detailed design
 - 6.1.1 Module 1 detail
 - 6.1.2 Module 2 detail
 - 6.2 Data detailed design
 - 6.2.1 Data entry 1 detail
 - 6.2.2 Data entry 2 detail

Design View	Scope	Entity attribute	Example representation
Decomposition description	Partition of the system into design entities	Identification, type purpose, function, subordinate	Hierarchical decomposition diagram, natural language
Dependency description	Description of relationships among entities of system resources	Identification, type, purpose, dependencies, resources	Structure chart, data flow diagrams, transaction diagrams
Interface description	List of everything a designer, developer, tester needs to know to use design entities that make up the system	Identification, function, interfaces	Interface files, parameter tables
Detail description	Description of the internal design details of an entity	Identification, processing, data	Flow charts, PDL etc.



CODING

MS. CHESHTAA BHARDWAJ

**ASSISTANT PROFESSOR, DEPT. OF CEA,
GLA UNIVERSITY, MATHURA**

WHAT IS CODING?

- The coding is the process of transforming the design of a system into a computer language format.
- This coding phase of software development is concerned with software translating design specification into the source code.
- It is necessary to write source code & internal documentation so that conformance of the code to its specification can be easily verified.
- Coding is done by the coder or programmers who are independent people than the designer.
- The goal is not to reduce the effort and cost of the coding phase, but to cut to the cost of a later stage. The cost of testing and maintenance can be significantly reduced with efficient coding.

GOALS OF CODING

- To translate the design of system into a computer language format.
- To reduce the cost of later phases.
- Making the program more readable.

CHARACTERISTICS OF PROGRAMMING LANGUAGE



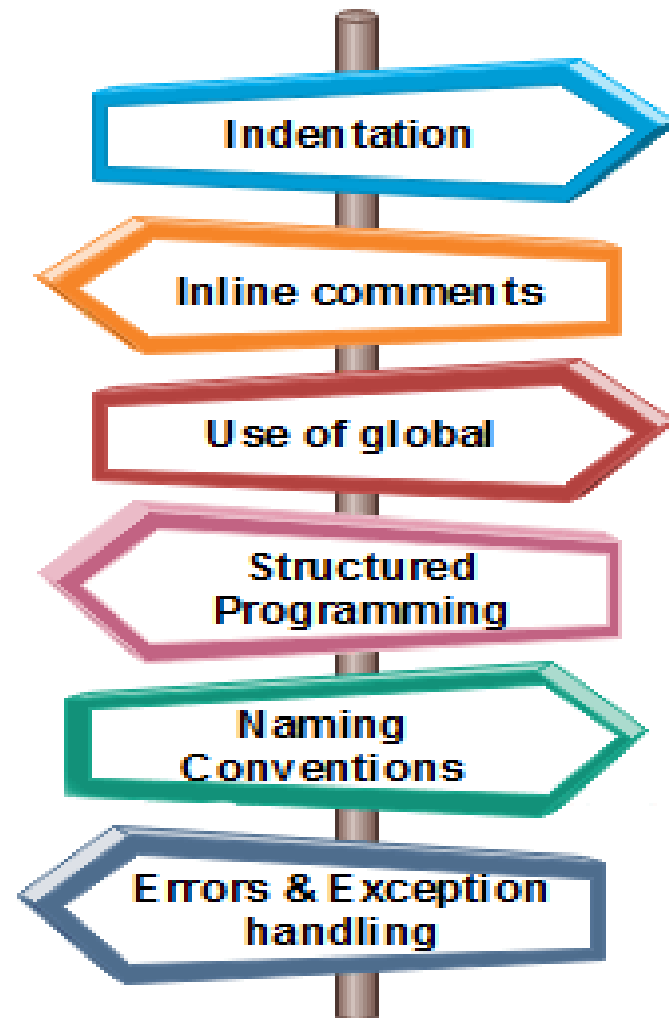
Contd.

- **Readability**: A good high-level language will allow programs to be written in some methods that resemble a quite-English description of the underlying functions. The coding may be done in an essentially self-documenting way.
- **Portability**: High-level languages, being virtually machine-independent, should be easy to develop portable software.
- **Generality**: Most high-level languages allow the writing of a vast collection of programs, thus relieving the programmer of the need to develop into an expert in many diverse languages.
- **Brevity**: Language should have the ability to implement the algorithm with less amount of code. Programs mean in high-level languages are often significantly shorter than their low-level equivalents.

Contd.

- **Error checking:** A programmer is likely to make many errors in the development of a computer program. Many high-level languages invoke a lot of bugs checking both at compile-time and run-time.
- **Cost:** The ultimate cost of a programming language is a task of many of its characteristics.
- **Quick translation:** It should permit quick translation.
- **Efficiency:** It should authorize the creation of an efficient object code.
- **Modularity:** It is desirable that programs can be developed in the language as several separately compiled modules, with the appropriate structure for ensuring self-consistency among these modules.
- **Widely available:** Language should be widely available, and it should be feasible to provide translators for all the major machines and all the primary operating systems.

CODING STANDARDS



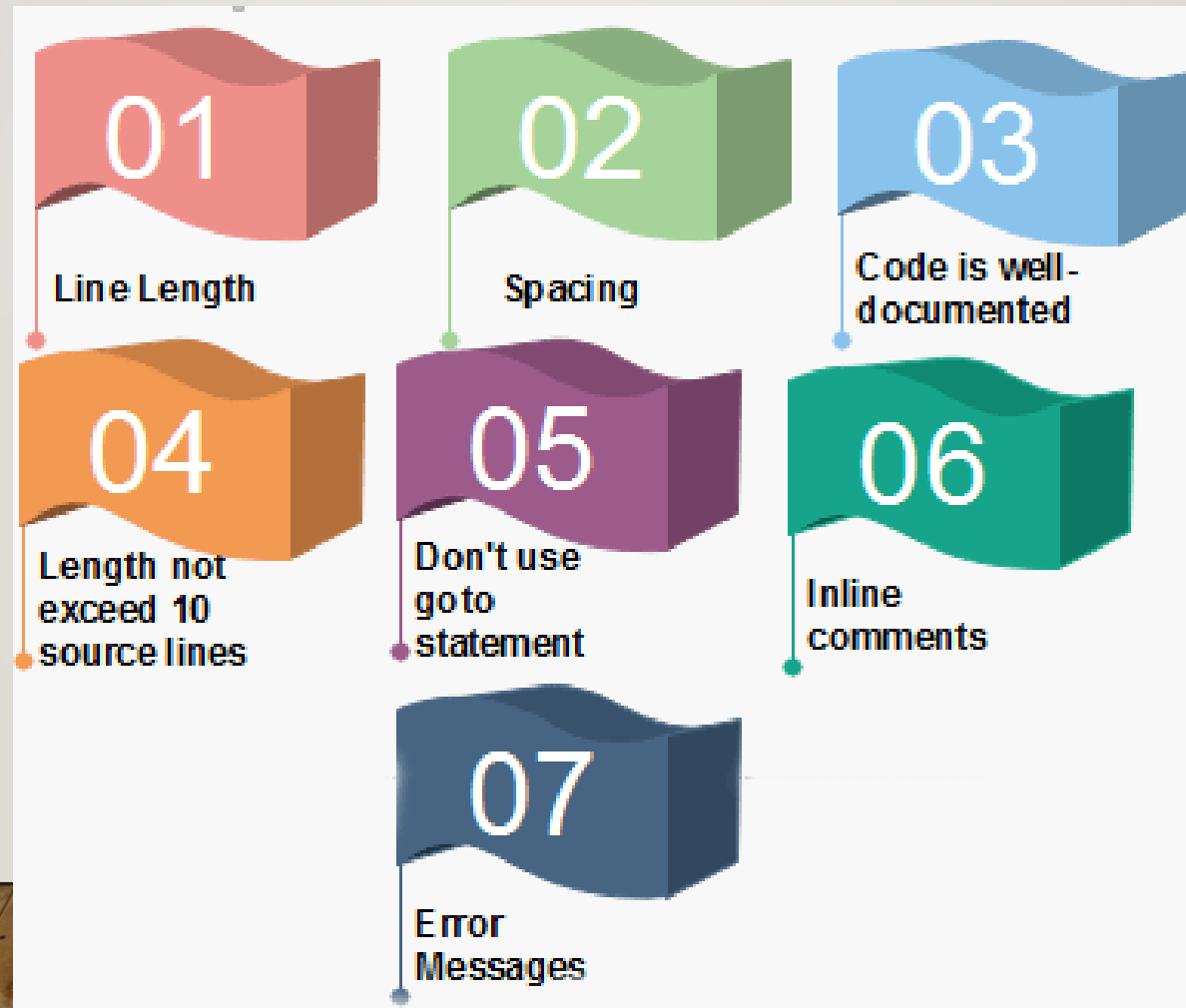
Contd.

- **Indentation:** Proper and consistent indentation is essential in producing easy to read and maintainable programs. Indentation should be used to:
 - Emphasize the body of a control structure such as a loop or a select statement.
 - Emphasize the body of a conditional statement
 - Emphasize a new scope block
- **Inline comments:** Inline comments analyze the functioning of the subroutine, or key aspects of the algorithm shall be frequently used.
- **Rules for limiting the use of global:** These rules file what types of data can be declared global and what cannot.

Contd.

- **Structured Programming:** Structured (or Modular) Programming methods shall be used.
- **Naming conventions for global variables, local variables, and constant identifiers:** A possible naming convention can be that global variable names always begin with a capital letter, local variable names are made of small letters, and constant names are always capital letters.
- **Error return conventions and exception handling system:** Different functions in a program report the way error conditions are handled should be standard within an organization. For example, different tasks while encountering an error condition should either return a 0 or 1 consistently.

CODING GUIDELINES



Contd.

- **Line Length:** It is considered a good practice to keep the length of source code lines at or below 80 characters. Lines longer than this may not be visible properly on some terminals and tools. Some printers will truncate lines longer than 80 columns.
- **Spacing:** The appropriate use of spaces within a line of code can improve readability.

Example:

Bad: `cost=price+(price*sales_tax)`
 `fprintf(stdout,"The total cost is %5.2f\n",cost);`

Better: `cost = price + ( price * sales_tax )`
 `fprintf (stdout,"The total cost is %5.2f\n",cost);`

Contd.

- **The code should be well-documented:** As a rule of thumb, there must be at least one comment line on the average for every three-source line.
- **The length of any function should not exceed 10 source lines:** A very lengthy function is generally very difficult to understand as it possibly carries out many various functions. For the same reason, lengthy functions are possible to have a disproportionately larger number of bugs.
- **Do not use goto statements:** Use of goto statements makes a program unstructured and very tough to understand.
- **Inline Comments:** Inline comments promote readability.
- **Error Messages:** Error handling is an essential aspect of computer programming. This does not only include adding the necessary logic to test for and handle errors but also involves making error messages meaningful.

E-BOOKS

- **Software Engineering- A Practitioner's Approach** by **Roger S. Pressman**
- **Software Engineering** Ninth Edition by **Ian Sommerville**
- **Software Engineering** Third Edition by **K.K Aggarwal & Yogesh Singh**